DM587 - Assignment 1

Vectors and matrices in Python

Deadline: Tuesday 10 November 2026, 23:59:59 Copenhagen time (CET). GitLab must receive your successful push to **main before Wednesday 11 November 2026, 00:00:00 CET**. Pushes received at or after that cutoff are not considered for marking.

Your highest Assignment 1 score from pushes received before the deadline will be used. See the final page for the deadline details.

Total available: 100 points. Implement the supplied vector and matrix classes yourself, using Python lists, dictionaries, sets, loops and your own functions. Read this sheet together with the function docstrings in the starter files.

What this assignment is about

Build your own small linear algebra library. You will connect vector and matrix operations to Python classes and operators, so expressions such as `u + v`, `u * v` and `A * B` perform the correct mathematical computations. You will also implement the same ideas using sparse storage, where entries that are not stored represent zero. The aim is to understand both how the computations work and how their data representation affects the implementation.

The graded work is your implementation of these operations. `benchmark.py` is only an extra tool for comparing speed and memory use; you do not have to run it and it earns no points.

Assignment 1 in your private GitLab project

Assignment 1 is in your existing private project, named `YOUR-USERNAME-repo` in the `dm587-2026` group. Open its `asg-vecmat` folder. Stage 1 explains how to find and download it. Use the separate student Git guide for general Git setup and commands.

```
YOUR-USERNAME-repo/
  .gitlab-ci.yml          automatic grading configuration
  asg-vecmat/
    README.md
    vec.py               EDIT: dense vectors
    mat.py               EDIT: dense matrices
    vec_sparse.py        EDIT: sparse vectors
    mat_sparse.py        EDIT: sparse matrices
    benchmark.py         leave unchanged
  ... other course files
```

The four implementation files belong directly inside `asg-vecmat/` at the project root. Do not create another `src/` folder or another copy of `asg-vecmat/`. Keep filenames, class names, function names, parameters and supplied data representations unchanged. Do not edit the grading configuration or other instructor-provided grading files.

Important rule: no NumPy or SciPy in your implementation

Do not import or call NumPy or SciPy in any of the four implementation files. A violation gives 0/100 for the entire assignment. This includes imports inside a function or class, aliases such as `import numpy as np`, and imports such as `from scipy.sparse import ...`. Write the computations yourself rather than delegating them to these libraries.

An initial check scans all four files before the grading tests run. If it finds a prohibited import, grading stops and feedback identifies the file and line. Remove the import, correct your implementation and push again before the deadline. Passing this check earns no extra points.

`benchmark.py` is the exception: its supplied NumPy/SciPy imports are intentional. Leave it unchanged. Running the benchmark is not required to submit and earns no points.

Question 1 - Dense vectors

38 points - file: `asg-vecmat/vec.py`

Complete the unfinished functions used by the provided `Vec` class. A dense vector stores its entries in a list named `store`; `size` is its length. Indices start at zero.

Implement `getitem`, `setitem`, `equal`, `add`, `dot`, `scalar_mul` and `neg`. The class already connects them to Python operators. Subtraction is provided through addition and negation; make those operations work correctly rather than adding a differently named submission function.

Graded operation	Required behaviour	Points
Vector equality	Compare corresponding entries for compatible vectors.	8
Vector addition	Return a new vector containing entrywise sums.	8
Vector subtraction	Return a new vector containing entrywise differences.	7
Dot product	Return the sum of the products of corresponding entries.	8
Scalar multiplication	Return a new vector with every entry scaled by the scalar.	7
Question 1 total		38

Reading an entry returns its value. Assigning an entry updates an existing position without extending the vector. Negation returns the vector with every sign reversed. These supporting operations have no separate points, but other operations depend on them.

Except for explicit item assignment, arithmetic operations must leave their input objects unchanged. Return the specified vector or scalar; printing a result does not replace returning it. Follow the assertions and examples in each function's docstring.

Behaviour to aim for

```
u = Vec([1, 2])
v = Vec([3, 4])
u + v == Vec([4, 6])       # True
u * v                     # 11
2 * u == Vec([2, 4])       # True
u == Vec([1, 2])           # True: u was not changed
```

These are examples of the required behaviour, not a complete solution. Add your own checks, including zero values, negative values and unequal vectors.

Question 2 - Dense matrices

52 points - file: `asg-vecmat/mat.py`

Complete the unfinished functions used by the supplied `Mat` class. A dense matrix stores rows as lists inside `store`; `size` is `(number_of_rows, number_of_columns)`. Access entry `(i, j)` with `M[i, j]`.

Implement `getitem`, `setitem`, `equal`, `add`, `scalar_mul`, `transpose`, `vector_matrix_mul`, `matrix_vector_mul` and `matrix_matrix_mul`. Use your `Vec` implementation from Question 1 where a vector result is required.

Graded operation	Required behaviour	Points
Matrix equality	Compare corresponding entries for compatible matrices.	7
Matrix addition	Return the entrywise sum of two matrices.	7
Scalar multiplication	Scale every matrix entry.	7
Transpose	Exchange rows and columns.	7
Vector times matrix	Return a vector; its length equals the matrix's column count.	8
Matrix times vector	Return a vector; its length equals the matrix's row count.	8
Matrix times matrix	Return the matrix product for compatible dimensions.	8
Question 2 total		52

`getitem` and `setitem` have no separate point allocation but must work correctly. Except for explicit item assignment, leave the input objects unchanged and return a new result. Do not replace matrix multiplication with entrywise multiplication.

Dimension checks

For a matrix with `m` rows and `n` columns:

- Its transpose has `n` rows and `m` columns.
- Matrix times vector requires a vector of length `n` and returns length `m`.
- Vector times matrix requires a vector of length `m` and returns length `n`.
- Multiplying an `m` by `n` matrix by an `n` by `p` matrix returns an `m` by `p` matrix.

Use the function docstrings as the specification for indexing, return types, operator behaviour and worked examples. Keep the provided `from vec import Vec` connection between the dense files.

Question 3 - Sparse vectors and matrices

10 points - files: `asg-vecmat/vec_sparse.py` **and** `asg-vecmat/mat_sparse.py`

Complete the sparse versions of the operations in Questions 1 and 2. These files also define classes named `Vec` and `Mat`; keep those names. The sparse matrix file already imports its vector class from `vec_sparse.py`.

A sparse vector has a label set `D` and a dictionary `f`. A sparse matrix has a pair of label sets `D` (rows, columns) and a dictionary `f` whose keys are `(row_label, column_label)` pairs. A missing dictionary entry represents zero. Labels are not necessarily consecutive integers and need not have a natural ordering.

Implement the unfinished vector and matrix functions, including indexing, item assignment, equality, addition, negation/scaling and the products provided by the templates. Equality must compare mathematical values: an omitted entry and an explicitly stored zero represent the same value. Arithmetic must not alter its inputs.

Graded operation	Points
Sparse matrix times sparse matrix	10
Question 3 total	10

The sparse grading section allocates its 10 points to matrix multiplication. The supporting sparse functions have no separate point allocation, but are part of the implementation task and may be needed by multiplication and comparison. Follow all supplied docstrings.

Marking summary

Question	Points
1. Dense vectors	38
2. Dense matrices	52
3. Sparse implementation	10
Total	100

The online grader reports dense results out of 90, sparse results out of 10, and their combined total out of 100. A failed graded check loses the points shown for that operation; one implementation mistake can affect several checks. The import rule overrides both sections and gives 0/100. There are no additional benchmark or speed bonus points. Grading tests have execution limits, so infinite loops or excessively slow code can fail.

Stages 1 and 2 - Get ready and work locally

The workflow: get the template once, edit and check your code, submit it, then read your grade. Repeat the last three stages whenever you improve your solution.

Stage 1 - Get the Assignment 1 template

Sign in at **https://gitlab.sdu.dk**. Open your assigned private course project named `YOUR-USERNAME-repo` in the `dm587-2026` group. Replace `YOUR-USERNAME` with your actual GitLab username. The project address follows this pattern:

```
https://gitlab.sdu.dk/dm587-2026/YOUR-USERNAME-repo
```

You can also use the private project link already supplied by the instructor. Use your own project, not another student's profile or repository. You do not create a new project for Assignment 1.

- On the project's file page, select **main** and open `asg-vecmat`. This is the Assignment 1 template folder.

- It must contain `README.md`, `vec.py`, `mat.py`, `vec_sparse.py`, `mat_sparse.py` and `benchmark.py`. Read the README and the function docstrings before editing.

- If the project is inaccessible or `asg-vecmat` is missing, contact the instructor before starting.

To download it, follow **Stage 1 of the separate Mac/Linux or Windows Git guide**, using **Code > Clone with HTTPS** from this private project. Use `assignment-1` as your local folder name. The downloaded project must contain `asg-vecmat` directly inside that folder. Keep any existing Assignment 0 working folder and files intact.

If you have already downloaded Assignment 1 and can see `asg-vecmat` locally, continue to Stage 2. You do not need to clone again or routinely download updates. The instructor will give you specific instructions if an update becomes necessary.

Stage 2 - Write, save and check your code

Edit the four implementation files inside `asg-vecmat/`. Keep `benchmark.py` unchanged. **Save your files in your editor before running checks or submitting.**

Use Python 3.8 or newer. Run these commands from the **project root**, the folder containing `asg-vecmat/`:

```
python3 --version
python3 -m doctest -v asg-vecmat/vec.py
python3 -m doctest -v asg-vecmat/mat.py
python3 -m doctest -v asg-vecmat/vec_sparse.py
python3 -m doctest -v asg-vecmat/mat_sparse.py
```

On Windows, replace `python3` with `py -3` if that is your Python command. These checks run the public examples. Failures show expected and actual results. Passing them does not guarantee full marks online. You do not need NumPy, SciPy or pygrade installed locally for these checks.

At this stage your changes are still on your computer. Running Python checks does not submit your work.

Stages 3 and 4 - Submit and read your grade

Stage 3 - Submit your saved code

In a terminal at the **project root**, run these four commands, one line at a time:

```
git add asg-vecmat/vec.py asg-vecmat/mat.py
git add asg-vecmat/vec_sparse.py asg-vecmat/mat_sparse.py
git commit -m "Submit Assignment 1"
git push origin main
```

The two `git add` lines select the four solution files. `git commit` saves a local revision. **A successful** `git push` **sends it to GitLab and starts grading.** You may reuse the same commit message for later submissions.

If a command fails, use the separate student Git guide or ask the instructor. Do not upload a ZIP, screenshots, printed output or a separate solution file. Your solution is the code in the four named files.

Stage 4 - Check the grade in GitLab

Open your private project in the browser. On branch **main**, check that `asg-vecmat/` contains your latest saved code. Then:

- Open **Build > Pipelines** and select the pipeline for your latest commit.
- Open `asg_assignment1` and wait until the job finishes.
- Read the log's **Assignment 1: Grade** line for your total out of 100.

The same job prints dense results out of 90, sparse results out of 10 and details of failed checks. Grading starts automatically after a successful push. Read the result in **asg_assignment1**.

```
Example: full marks
Verified dense grade: 90.00/90.00
Verified sparse grade: 10.00/10.00
Assignment 1: Grade: 100.00/100.00

Example: the 8-point dot-product check fails
Verified dense grade: 82.00/90.00
Verified sparse grade: 10.00/10.00
Assignment 1: Grade: 92.00/100.00
```

Your score depends on your own implementation. Feedback names failed tests and points deducted, with available error details. For example, `test_vec_dot` identifies the dot-product check. A job marked **Failed** can still contain a valid grade: deductions make the job fail, and your pushed code has still been received.

A prohibited NumPy/SciPy import gives **0/100** and stops correctness testing. The feedback identifies the file and line.

To improve your solution: repeat Stage 2, Stage 3 and Stage 4 before the deadline. Local check results appear in your terminal; the online grade appears in GitLab.

Assignment 1 - Deadline and final score

Which submission counts

The cutoff is **Wednesday 11 November 2026, 00:00:00 CET (Europe/Copenhagen)**. Your push must be successfully received by GitLab before the cutoff.

- We use your **highest Assignment 1 score across all successful pushes to main received before the cutoff** in your assigned private project. A later, lower-scoring submission does not reduce an earlier eligible score.

- Pushes received at or after 00:00:00 on Wednesday 11 November are late and are excluded from marking, even if GitLab still accepts and grades them.

- A file saved locally, a local commit, a rejected push or a push to a different branch does not count as a submission to main.

- GitLab's push-receipt time determines eligibility. Your computer's clock and the author/commit timestamps do not.

- If an on-time push waits in a grading queue, its score remains eligible when grading finishes after the cutoff. Eligibility depends on when GitLab received the push, not when its job finishes.

- If a submission has a grading setup error and no valid score, contact the instructor. A setup error requires review and is not a numerical zero.

Allow time for network or Git problems before the deadline. Scores from late pushes are excluded, even if they are higher.

Find saved feedback or report a grading problem

The job log is sufficient for checking the grade. You can also browse or download the job's **artifacts** and open `feedback/assignment1/summary.txt`. Detailed reports are under `feedback/assignment1/dense/`, `sparse/` and `import-check/`. Feedback is retained even when the job fails.

A waiting job may be queued; refresh the page. If a successful push produces no pipeline, or the job says **Score unavailable** / **grading setup error**, contact the instructor with your project URL, commit identifier and pipeline/job URL. A setup error is different from a numerical grade with deductions.